

PlacementIQ

Enterprise Campus Recruitment Management System — Full Specification

Architecture: FastAPI + React 18	Database: PostgreSQL (steps_db)	Version: 2.0 Python Edition	Date: September 2026
---	--	------------------------------------	-----------------------------

1. Executive Summary

PlacementIQ is an end-to-end automated Campus Placement and Recruitment Management System designed to streamline hiring workflows for educational institutions, corporate recruiters, and student candidates. The platform bridges the communication gap between university placement cells and corporate talent acquisition teams by unifying job publication, applicant tracking, multi-stage assessment management, AI-driven candidate keyword screening, in-app communication, and real-time operational analytics into a cohesive application.

Key Value Propositions:

- **Automated Multi-Stage Pipeline:** Enables recruiters to build customized multi-round evaluation tracks (MCQ assessments, live coding challenges, technical interviews, HR rounds).
- **AI-Powered Screening Engine:** Uses keyword matching algorithms against candidate resumes, skill sets, and cover letters to immediately rank applicants.
- **Complete Operational Audit Trail:** Logs recruiter actions (job posting, edits, pauses, resumes, and deletions) via structured activity history logs.
- **High-Performance Python Architecture:** Migrated to FastAPI and SQLAlchemy for async capabilities, type-safe request validation, and high throughput.

2. Technical Architecture & Stack

PlacementIQ operates on a modern decoupled client-server architecture. The React single-page frontend communicates with the Python FastAPI REST backend over HTTP REST APIs secured by Bearer JWT authentication tokens.

Layer	Technology / Framework	Key Role & Responsibility
Frontend SPA	React 18, TypeScript, Vite	Responsive client user interface with dynamic routing, reactive state, and modal management.
UI Styling	Tailwind CSS, Lucide Icons	Modern styling framework and icon components.
API Client	Axios, Custom Service Layer	Encapsulates endpoints, injects Bearer JWT headers, and handles response handling.
Backend Core	Python 3.13, FastAPI	High-performance ASGI REST backend offering auto-generated Swagger docs and async execution.
Validation	Pydantic v2	Strict input/output request schema validation and data parsing.
Database ORM	SQLAlchemy 2.0	ORM with eager loading (<code>selectinload</code>), entity relationships, and cascades.
Auth System	PyJWT, Native Bcrypt	Stateless JWT token issuance/verification and salted password hashing.

Database Engine	PostgreSQL 14+ (`steps_db`)	Relational data store hosted on port 5431 with foreign key integrity constraints.
ASGI Server	Uvicorn	Lightning-fast ASGI web server hosting the FastAPI application.

3. Multi-Role User Hierarchy

PlacementIQ implements Role-Based Access Control (RBAC) to enforce operational boundaries across three primary roles:

User Role	Access Scope	Permissions & Core Functions
STUDENT	Candidate Portal	• View active open job postings. • Submit applications with custom cover letters and answers. • Track application statuses (`APPLIED`, `SHORTLISTED`, `OFFERED`, `REJECTED`). • Respond to active job offers (`ACCEPTED` / `DECLINED`). • Edit candidate profile (Skills, CGPA, Branch, Experience, Projects).
RECRUITER	Employer Dashboard	• Create and publish job openings with custom screening questions. • Define multi-round interview pipelines (MCQ, Coding, Technical, HR). • Trigger AI-assisted candidate resume and keyword screening. • Update applicant statuses across rounds. • Manage job posting lifecycle (Pause, Resume, Edit, Delete). • Access job activity audit logs and candidate communication chats.
ADMIN	System Admin Panel	• Review and verify pending recruiter account registrations. • Access global placement analytics (Placement Rate %, Total Placements, Open vs Closed Jobs). • Monitor university-wide application statistics.

4. Key Business Workflows

A. Job Creation & Hiring Pipeline Configuration

1. Recruiter submits job metadata (Title, Description, Salary Range, Location, Start/End Dates).
2. Recruiter configures screening questions for candidates to fill out during application.
3. Recruiter adds ordered evaluation rounds (e.g., Round 1: Online MCQ, Round 2: Coding Assessment, Round 3: HR Interview).
4. System saves job entity, binds rounds, and records a **POSTED** event in JobActivityLog.

B. Candidate Discovery & Application Submission

1. Student browses open listings on Candidate Dashboard.
2. Student opens job application modal, inputs cover letter, answers custom screening questions, and submits.
3. System validates constraints (duplicate application check, application window validation).
4. System creates Application entity with initial status APPLIED.

C. AI Keyword Screening & Candidate Shortlisting

1. Recruiter triggers AI Screening by submitting target skill keywords (e.g., 'Python, React, PostgreSQL').
2. Backend calculates keyword presence across candidate resume skills, experience, and cover letter text.

- 3. System computes a percentage match score (0–100%) and generates structured feedback.
- 4. Recruiter updates candidate status to SHORTLISTED, automatically advancing candidate into Round 1 of CandidateProgress.

D. Job Lifecycle Management & Operational Audit Logging

- 1. Recruiter toggles job state to Paused (isOpen = False). System logs **PAUSED** activity.
- 2. Recruiter updates job details. System logs **EDITED** activity.
- 3. Recruiter deletes job. System logs **DELETED** activity and performs cascade cleanup.

5. Database Entities & Schema

PlacementIQ relies on a relational model implemented in PostgreSQL:

Entity Model	Primary Key & Foreign Keys	Key Attributes & Data Types
User	`id` (UUID)	`email`, `passwordHash`, `fullName`, `role` (Enum: `STUDENT`, `RECRUITER`, `ADMIN`), `createdAt`.
StudentProfile	`id`, FK: `userId` -> User	`college`, `branch`, `graduationYear`, `skills`, `cgpa`, `experience`, `projects`.
RecruiterProfile	`id`, FK: `userId` -> User	`companyName`, `designation`, `status` (Enum: `PENDING`, `APPROVED`, `REJECTED`).
Job	`id`, FK: `recruiterProfileId`	`title`, `description`, `requirements`, `salaryRange`, `jobType`, `location`, `isOpen`, `questions` (JSON), `applyStartDate`, `applyEndDate`.
InterviewRound	`id`, FK: `jobId`	`title`, `type`, `format`, `description`, `instructions`, `order`, `mcqDuration`, `codingDuration`.
Application	`id`, FK: `jobId`, `studentProfileId`	`status` (Enum), `coverLetterText`, `answers` (JSON), `aiScreeningScore`, `aiScreeningFeedback`, `appliedAt`.
CandidateProgress	`id`, FK: `applicationId`, `roundId`	`status` (`PENDING`, `QUALIFIED`, `REJECTED`), `feedback`, `meetLink`, `isMeetLinkPublished`.
JobActivityLog	`id`, FK: `recruiterProfileId`	`jobId`, `jobTitle`, `action` (`POSTED`, `EDITED`, `PAUSED`, `RESUMED`, `DELETED`), `timestamp`, `details`.
RoundMessage	`id`, FK: `jobId`, `roundId`	`senderId`, `senderName`, `content`, `createdAt`.

6. Backend API Architecture

FastAPI Router Endpoints Summary:

Method	Endpoint Path	Auth & Scope	Description
POST	/api/auth/signup	Public	Registers User + Student or Recruiter Profile.
POST	/api/auth/login	Public	Authenticates credentials & returns JWT token.
GET	/api/auth/profile	Authenticated	Fetches authenticated user & profile details.
PUT	/api/auth/profile	Authenticated	Updates candidate skills or recruiter company info.
GET	/api/jobs/	Public	Lists all active/open jobs for candidate view.
GET	/api/jobs/me	RECRUITER	Fetches recruiter's posted jobs with application counts.
GET	/api/jobs/history	RECRUITER	Fetches recruiter's activity audit logs.
POST	/api/jobs/	RECRUITER	Posts new job opportunity and logs POSTED event.
GET	/api/jobs/{id}	Public	Gets job details with screening questions & rounds.
PUT	/api/jobs/{id}	RECRUITER	Updates job info or toggles open/paused state.
DELETE	/api/jobs/{id}	RECRUITER	Deletes job posting with cascade cleanup.
POST	/api/jobs/{id}/rounds	RECRUITER	Saves interview round pipeline definitions.
GET / POST	/api/jobs/{id}/messages	Authenticated	Fetches and posts in-app candidate messages.
POST	/api/applications/{jobId}	STUDENT	Submits application with cover letter & answers.
GET	/api/applications/me	STUDENT	Gets candidate application tracking history.
GET	/api/applications/job/{id}	RECRUITER	Gets all candidate applications submitted for a job.
PATCH	/api/applications/{id}/status	RECRUITER	Updates candidate status (SHORTLISTED, OFFERED).
PATCH	/api/applications/{id}/respond	STUDENT	Student accepts or declines active offer.
POST	/api/applications/job/{id}/ai-screen	RECRUITER	Executes AI keyword screening on applicants.
GET	/api/analytics/student	STUDENT	Student metrics & active job counts.
GET	/api/analytics/recruiter	RECRUITER	Recruiter jobs chart data & status counts.
GET	/api/analytics/admin	ADMIN	System-wide metrics & placement rate %.

7. Repository Structure

PlacementIQ/	
■■■■ backend/	
■ ■■■■ venv/	# Python 3.13 Virtual Environment
■ ■■■■ models.py	# SQLAlchemy ORM Database Models (User, Job, Application, etc.)
■ ■■■■ database.py	# Database engine & SessionLocal provider
■ ■■■■ auth.py	# JWT token generation, verification & bcrypt password utilities
■ ■■■■ main.py	# FastAPI application entry point & CORS configuration
■ ■■■■ routes/	
■ ■ ■■■■ auth.py	# Auth & profile management routes
■ ■ ■■■■ jobs.py	# Jobs CRUD, rounds config, activity history & messaging
■ ■ ■■■■ applications.py	# Job applications, recruiter candidate management & AI screening
■ ■ ■■■■ analytics.py	# Real-time metrics for Student, Recruiter & Admin
■ ■ ■■■■ rounds.py	# Evaluation round management
■ ■ ■■■■ admin.py	# Admin profile approvals & system controls
■ ■■■■ .env	# Environment variables (DB Connection: localhost:5431/steps_db)

```
■
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ components/      # Reusable UI components & modals
■   ■   ■■■ pages/           # View pages (StudentDashboard, RecruiterDashboard, AdminDashboard, Auth)
■   ■   ■■■ services/
■   ■   ■   ■■■ api.ts       # Axios API service encapsulation layer
■   ■   ■■■ App.tsx          # Main React Router setup
■   ■   ■■■ main.tsx         # React application entry point
■   ■■■ package.json         # Node.js dependencies (React, Vite, Lucide Icons, Axios)
■   ■■■ .env                 # Frontend environment variables (VITE_API_URL=http://127.0.0.1:8000/api)
■
■■■ README.md                # Project setup & usage instructions
```